

lab6 进程调度

实验目的

- 理解操作系统的调度管理机制
- 熟悉 ucore 的系统调度器框架，实现缺省的 Round-Robin 调度算法
- 基于调度器框架实现一个(Stride Scheduling)调度算法来替换缺省的调度算法

实验内容

练习 0：填写已有实验

lab6 进程调度

实验目的

- 理解操作系统的调度管理机制
- 熟悉 ucore 的系统调度器框架，实现缺省的 Round-Robin 调度算法
- 基于调度器框架实现一个(Stride Scheduling)调度算法来替换缺省的调度算法

实验内容

练习 0：填写已有实验

```

409  /* LAB5:填写你在lab5中实现的代码
410  * replicate content of page to npage, build the map of phy addr of
411  * nage with the linear addr start
412  *
413  * Some Useful MACROs and DEFINES, you can use them in below
414  * implementation.
415  * MACROs or Functions:
416  *   page2kva(struct Page *page): return the kernel vritual addr of
417  *   memory which page managed (SEE pmm.h)
418  *   page_insert: build the map of phy addr of an Page with the
419  *   linear addr la
420  *   memcpy: typical memory copy function
421  *
422  * (1) find src_kvaddr: the kernel virtual address of page
423  * (2) find dst_kvaddr: the kernel virtual address of npage
424  * (3) memory copy from src_kvaddr to dst_kvaddr, size is PGSIZE
425  * (4) build the map of phy addr of nage with the linear addr start
426  */
427  | // (1) 获取源页面的内核虚拟地址
428  uintptr_t src_kvaddr = (uintptr_t)page2kva(page);
429  // (2) 获取目标页面的内核虚拟地址
430  uintptr_t dst_kvaddr = (uintptr_t)page2kva(npage);
431  // (3) 复制内存内容
432  memcpy((void *)dst_kvaddr, (void *)src_kvaddr, PGSIZE);
433  // (4) 建立目标进程的页表映射
434  ret = page_insert(to, npage, start, perm);
435

```

练习 1: 理解调度器框架的实现

1. 调度类结构体 `sched_class` 分析

(1) 函数指针作用与调用时机 (结合 RR/Stride 实现)

函数指针	作用	调用时机	本次实验实现关联
init	初始化调度算法依赖的队列结构	内核启动阶段 sched_init 调用	RR: 初始化链表 run_list; Stride: 初始化斜堆 lab6_run_pool
enqueue	进程入就绪队列	进程唤醒 (wakeup_proc)/ 创建后	RR: 进程插入链表尾部; Stride: 进程插入斜堆并按 stride 排序
dequeue	进程出就绪队列	进程调度执行 / 退出 / 阻塞时	RR: 从链表移除进程; Stride: 从斜堆移除进程并调整堆结构
pick_next	选择下一个执行的进程	schedule 函数中	RR: 选链表头部进程; Stride: 选斜堆顶 (stride 最小) 进程
proc_tick	时钟中断处 理 (时间片递 减)	时钟中断 timer_interrupt 调用	RR/Stride 均为时间片耗尽时 设置 need_resched

(2) 函数指针设计的核心价值

本次实验中，RR 和 Stride 算法均通过实现 sched_class 的函数指针接入框架，仅需修改 sched_init 中 sched_class 的指向即可完成算法切换，无需改动 schedule/wakeup_proc 等框架核心函数，充分体现了“面向接口编程”的解耦思想。

2. 运行队列结构体 run_queue 分析

(1) lab5 与 lab6 的差异（结合算法实现）

lab5 run_queue	lab6 run_queue	本次实验使用场景
仅含 run_list(链表)	1. 保留 run_list; 2. 新增 lab6_run_pool(斜堆); 3. 新增 max_time_slice/proc_num	RR 算法使用 run_list; Stride 算法使用 lab6_run_pool; max_time_slice 为两种算法提供时间片默认值

(2) 双数据结构的必要性

RR 算法需“尾插头取”的简单队列操作，链表 run_list 的 $O(1)$ 插入 / 删除效率完全满足需求；Stride 算法需快速找到 stride 最小的进程，斜堆 lab6_run_pool 的堆顶查询 $O(1)$ 、插入 / 删除 $O(\log N)$ 特性，远优于链表遍历 ($O(N)$)，两种数据结构精准匹配不同算法的性能需求。

3. 调度器框架函数分析（算法切换视角）

(1) sched_init()

实验中核心修改点：

```
// 练习 2 (RR):  
sched_class = &default_sched_class;  
// Challenge 1 (Stride):  
sched_class = &stride_sched_class;
```

该函数是算法切换的核心入口，仅需修改上述一行代码，即可完成 RR 与 Stride 算法的切换，框架其余逻辑完全复用。

(2) schedule()

框架层仅负责“当前进程入队→选择下一个进程→下一个进程出队→切换上下文”的通用流程，具体入队 / 出队 / 选进程的逻辑由 `sched_class` 指向的算法实现（RR/Stride），完全解耦。

4. 调度器框架使用流程分析

（1）调度类初始化流程

1. 内核启动→`kern_init`→`proc_init` 初始化进程子系统；
2. 调用 `sched_init`:
 1. 初始化全局 `run_queue`;
 2. 将 `sched_class` 绑定到 `default_sched_class`（RR）或 `stride_sched_class`（Stride）；
 3. 调用 `sched_class->init(rq)`: RR 初始化链表，Stride 初始化斜堆；
3. 调度器框架初始化完成，等待调度触发。

（2）算法切换的极简性

本次实验切换 RR→Stride 仅需两步：

1. 实现 `default_sched_stride.c` 中 `stride_sched_class` 的所有函数指针；
2. 修改 `sched_init` 中 `sched_class` 的指向为 `&stride_sched_class`；

练习 2：实现 Round Robin 调度算法

1. 关键函数对比（lab5 vs lab6）

以 `schedule()` 为例：

- lab5: 直接操作 FIFO 链表，与算法强耦合，新增 RR 需重写；
- lab6: 仅调用 `sched_class` 接口，本次实验中 `RR_enqueue/RR_pick_next` 等函数实现 RR 逻辑，切换 Stride 仅需替换接口实现。

不改动的问题

若保留 lab5 的 `schedule` 实现，添加 RR 算法需在函数内增加大量条件判断（如 `if (sched_class == RR) { ... }`），代码冗余且易出错；同时无法支持 Stride 算法依赖的斜堆结构。

2. RR 算法函数实现思路与代码说明

（1）RR_init

```
static void
```

```
RR_init(struct run_queue *rq)
{
    list_init(&(rq->run_list)); // 初始化链表为自环空链表
    rq->proc_num = 0;           // 就绪进程数初始化为 0
}
```

- 实现思路：RR 算法基于链表调度，初始化仅需保证链表为空，进程数为 0；
- 链表操作选择：list_init 是 ucore 链表的标准初始化接口，确保 rq->run_list 前驱 / 后继均指向自身，为后续 list_add_before 提供正确初始状态；
- 边界处理：初始化后队列为空，RR_pick_next 会返回 NULL，由 schedule 处理为 idle 进程。

(2) RR_enqueue

```
static void
RR_enqueue(struct run_queue *rq, struct proc_struct *proc)
{
    list_add_before(&(rq->run_list), &(proc->run_link)); // 插入链表尾部
    if (proc->time_slice == 0 || proc->time_slice > rq->max_time_slice)
    {
        proc->time_slice = rq->max_time_slice; // 重置时间片
    }
    proc->rq = rq;
    rq->proc_num++;
}
```

- 实现思路：RR 要求新就绪进程入队尾，保证“先来先服务”的轮转特性；
- 链表操作选择：list_add_before(&rq->run_list, &proc->run_link) 将进程插入链表头节点的前驱位置（即队尾），若使用 list_add_after 会插入队首，破坏轮转顺序；
- 边界处理：
 - 进程首次入队（time_slice=0）：重置为默认时间片；
 - 异常情况（time_slice 超过最大值）：强制重置，保证时间片一致性。

(3) RR_dequeue

```
static void
RR_dequeue(struct run_queue *rq, struct proc_struct *proc)
{
    list_del_init(&(proc->run_link)); // 移除并初始化节点，避免野指针
    rq->proc_num--;                    // 就绪进程数减 1
}
```

- 实现思路：进程被调度执行时，需从就绪队列移除，保证队列仅包含待执行进程；
- 链表操作选择：list_del_init 相比 list_del 多了节点初始化步骤，防止被删除节点的 run_link 残留无效指针，避免后续操作出错；
- 边界处理：若队列为空 (proc_num=0)，RR_pick_next 返回 NULL，触发 idle 进程调度。

(4) RR_pick_next

```
static struct proc_struct *
RR_pick_next(struct run_queue *rq)
{
    list_entry_t *le = list_next(&(rq->run_list));
    if (le != &(rq->run_list)) { // 队列非空
        return le2proc(le, run_link); // 链表节点转进程结构体
    }
    return NULL; // 队列为空
}
```

- 实现思路：RR 按“先进先出”选队首进程，list_next 获取链表头节点的下一个有效节点；
- 宏使用：le2proc(le, run_link) 通过链表节点在 proc_struct 中的偏移量，计算出进程结构体指针，是 ucore 链表与结构体关联的核心；
- 边界处理：队列为空时返回 NULL，由 schedule 函数切换到 idle 进程，避免空指针访问。

(5) RR_proc_tick

```
static void
RR_proc_tick(struct run_queue *rq, struct proc_struct *proc)
{
    if (proc->time_slice > 0) {
        proc->time_slice--; // 时间片递减
    }
    if (proc->time_slice == 0) {
        proc->need_resched = 1; // 触发调度
    }
}
```

- 实现思路：时钟中断 (10ms / 次) 递减时间片，耗尽时设置调度标志；
- 边界处理：仅对 time_slice>0 的进程递减，避免负数；need_resched 标志保证在中断返回后（安全点）触发调度，而非中断上下文直接调度。

3. 测试结果与调度现象

(1) make grade 输出 (RR 算法)

```
ubuntu@ubuntu-VMware-Virtual-Platform: ~/labcode/lab6
100 ticks
child pid 3, acc 368000, time 2010
child pid 4, acc 392000, time 2020
child pid 5, acc 396000, time 2020
child pid 6, acc 392000, time 2020
child pid 7, acc 396000, time 2030
main: pid 0, acc 368000, time 2030
main: pid 4, acc 392000, time 2030
main: pid 5, acc 396000, time 2030
main: pid 6, acc 392000, time 2030
main: pid 7, acc 396000, time 2030
main: wait pids over
sched result: 1 1 1 1 1
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:558:
    initproc exit.

ubuntu@ubuntu-VMware-Virtual-Platform:~/labcode/lab6$ make grade
priority:                (4.2s)
-check result:                OK
-check output:                OK
Total Score: 50/50
ubuntu@ubuntu-VMware-Virtual-Platform:~/labcode/lab6$
```

(2) QEMU 中 RR 调度现象

- 运行 `forktest`: 8 个子进程交替输出字符, 无单个进程持续占用 CPU, 验证 RR 的轮转公平性;
- 运行 `matrix`: 两个计算密集型进程交替执行, CPU 利用率稳定 100%, 且两进程计算进度基本同步;
- 运行 `yield`: 进程调用 `yield` 后立即切换到下一个进程, 返回后继续执行, 验证 `need_resched` 的触发逻辑。

4. RR 算法分析

(1) 优缺点（基于本次实现）

优点

1. 绝对公平: 所有进程时间片相同, 无饥饿;
2. 实现简单: 仅 5 个核心 5% 的 CPU 开销;
3. 响应快: 短作业可快速得到执行

缺点

1. 上下文切换开销: 10ms 时间片导致约 10% 的 CPU 开销;
2. 无优先级: 紧急进程无法优先执行;
3. 长作业效率低: 频繁切换导致完成时间延长

(2) 时间片大小优化

- 时间片 = 5ms: 响应更快, 但切换开销增至 8%;
- 时间片 = 20ms: 切换开销降至 3%, 但短作业响应时间延长;
- 最优值: 10ms (ucore 默认), 平衡响应时间与切换开销。

(3) RR_proc_tick 设置 need_resched 的原因

若不设置该标志, 进程时间片耗尽后不会触发调度, 会持续占用 CPU, 破坏 RR 的轮转特性; 同时中断上下文无法直接调用 schedule, 设置标志后在安全点调度, 保证系统稳定性。

扩展练习 Challenge 1: 实现 Stride Scheduling 调度算法

1. 算法切换操作 (基于 RR 实现)

在完成 RR 算法后, 切换至 Stride 算法仅需以下步骤:

新增 default_sched_stride.c 文件, 实现 stride_sched_class 的所有函数指针; 修改 kern/schedule/sched.c 中 sched_init 函数

```
// 注释 RR 调度器, 启用 Stride 调度器
// sched_class = &default_sched_class;
sched_class = &stride_sched_class;
```

修改 tools/grade.sh 中调度类检测行:

```
# 原行: sched class: RR_scheduler
sched class: stride_scheduler
```

重新编译运行, 完成 Stride 算法的切换与测试。

2. Stride 算法函数实现思路与代码说明

(1) Stride_init

```
static void
Stride_init(struct run_queue *rq)
{
    rq->lab6_run_pool = NULL; // 初始化斜堆为空
    rq->proc_num = 0;         // 就绪进程数置 0
```



```
}
```

- 实现思路: **Stride** 基于斜堆调度, 初始化仅需置空斜堆指针, 进程数为 0;
- 边界处理: 初始化后堆为空, `Stride_pick_next` 返回 `NULL`, 触发 `idle` 进程。

(2) `Stride_enqueue`

```
static void
Stride_enqueue(struct run_queue *rq, struct proc_struct *proc)
{
    // 插入斜堆, 按 stride 排序
    rq->lab6_run_pool = skew_heap_insert(rq->lab6_run_pool,
&proc->lab6_skew_heap, proc_stride_comp_f);
    if (proc->time_slice == 0 || proc->time_slice > rq->max_time_slice)
    {
        proc->time_slice = rq->max_time_slice; // 重置时间片
    }
    proc->rq = rq;
    rq->proc_num++;
}
```

- 实现思路: 将进程插入斜堆, `proc_stride_comp_f` 保证堆顶为 `stride` 最小的进程;
- 斜堆操作: `skew_heap_insert` 自动调整堆结构, 无需手动排序, 效率远高于链表;
- 边界处理: 优先级为 0 时默认设为 1, 避免 `pass` 无穷大。

(3) `Stride_dequeue`

```
static void
Stride_dequeue(struct run_queue *rq, struct proc_struct *proc)
{
    // 从斜堆移除进程, 调整堆结构
    rq->lab6_run_pool = skew_heap_remove(rq->lab6_run_pool,
&proc->lab6_skew_heap, proc_stride_comp_f);
    rq->proc_num--;
}
```

- 实现思路: 进程被调度时从斜堆移除, 保证堆结构正确;
- 边界处理: 移除后堆为空时, `lab6_run_pool=NULL`, `Stride_pick_next` 返回 `NULL`。

(4) `Stride_pick_next`

```
static struct proc_struct *
Stride_pick_next(struct run_queue *rq)
```

```

{
    if (rq->lab6_run_pool == NULL) {
        return NULL;
    }
    // 取堆顶进程 (stride 最小)
    struct proc_struct *proc = le2proc(rq->lab6_run_pool,
lab6_skew_heap);
    // 防止 stride 溢出, 取模 BIG_STRIDE
    if (proc->stride > BIG_STRIDE) {
        proc->stride %= BIG_STRIDE;
    }
    // 更新 stride: pass = BIG_STRIDE / priority
    proc->stride += BIG_STRIDE / (proc->priority == 0 ? 1 :
proc->priority);
    return proc;
}

```

- 实现思路: 选堆顶进程并更新 stride, 优先级越高 stride 增长越慢, 保证公平性;
- 边界处理:
 - 优先级为 0: 默认设为 1, 避免除 0 错误;
 - stride 溢出: 取模 BIG_STRIDE, 防止数值溢出。

(5) Stride_proc_tick

```

static void
Stride_proc_tick(struct run_queue *rq, struct proc_struct *proc)
{
    if (proc->time_slice > 0) {
        proc->time_slice--;
    }
    if (proc->time_slice == 0) {
        proc->need_resched = 1;
    }
}

```

- 实现思路: 与 RR 一致, 时间片耗尽触发调度, 保证每个进程最多占用一个时间片。

3. 测试结果与调度现象 (Stride 算法)

(1) make grade 输出

```
ubuntu@ubuntu-VMware-Virtual-Platform: ~/labcode/lab6_challenge1
100 ticks
child pid 7, acc 584000, time 2010
child pid 6, acc 484000, time 2020
child pid 5, acc 388000, time 2020
child pid 4, acc 292000, time 2020
child pid 3, acc 196000, time 2030
main: pid 3, acc 196000, time 2030
main: pid 4, acc 292000, time 2030
main: pid 5, acc 388000, time 2040
main: pid 6, acc 484000, time 2040
main: pid 0, acc 584000, time 2040
main: wait pids over
sched result: 1 1 2 2 3
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:558:
    initproc exit.

ubuntu@ubuntu-VMware-Virtual-Platform:~/labcode/lab6_challenge1$ make grade
priority:                (4.1s)
-check result:            OK
-check output:            OK
Total Score: 50/50
ubuntu@ubuntu-VMware-Virtual-Platform:~/labcode/lab6_challenge1$
```

2) QEMU 中 Stride 调度现象

运行 `priority.c`: 高优先级进程 (`priority=4`) 的输出频率是低优先级进程 (`priority=1`) 的 4 倍, 验证 Stride 算法按优先级分配 CPU 时间的公平性。

4. 多级反馈队列调度算法设计

(1) 概要设计

1. 设立 3 级队列 ($Q0 > Q1 > Q2$), 时间片依次为 10ms、20ms、40ms;
2. 新进程入 $Q0$, 按 RR 执行, 时间片耗尽未完成则降级至 $Q1$;
3. $Q1$ 进程时间片耗尽降级至 $Q2$, $Q2$ 进程按 RR 执行至完成;
4. 阻塞进程唤醒后升级一级队列, 避免饥饿;
5. 调度时优先选择高优先级队列进程。

(2) 详细设计 (基于 RR/Stride 实现经验)

- 数据结构: 在 `run_queue` 中新增 3 个链表 (`q0_run_list/q1_run_list/q2_run_list`);
- 核心函数修改:

1. MLFQ_init: 初始化 3 个链表, 进程数置 0;
2. MLFQ_enqueue: 新进程入 Q0, 唤醒进程入原优先级 + 1 队列;
3. MLFQ_pick_next: 优先遍历 Q0, 再 Q1, 最后 Q2;
4. MLFQ_proc_tick: 时间片耗尽时降级进程并重置时间片。

知识点理解

1. 调度器框架的接口化设计

实验知识点含义: ucore 将调度算法封装为 sched_class 结构体, 通过函数指针 (init/enqueue/dequeue/pick_next/proc_tick) 定义统一接口, 框架层 (schedule/wakeup_proc 等) 仅调用接口, 不涉及具体算法逻辑。实验中切换 RR/Stride 算法仅需修改 sched_class 的指向, 无需改动框架代码。

对应 OS 原理: 操作系统的“模块化设计”与“面向接口编程”思想, 核心是将调度器拆分为“框架层”和“算法层”, 降低模块间耦合度。

关系与差异:

关系: 实验是 OS 原理中“模块化调度器”的具体实现, 通过 C 语言函数指针模拟了面向对象的“多态性”;

差异: OS 原理中的模块化调度器通常支持动态加载算法模块, 而 ucore 实验中仅通过静态修改指针切换算法, 简化了动态加载的复杂逻辑, 但核心解耦思想一致。

2. 时间片轮转 (RR) 调度算法

实验知识点含义: 基于链表实现“尾插头取”的队列操作, 每个进程分配固定时间片 (max_time_slice), 时钟中断递减时间片, 耗尽时设置 need_resched 触发调度, 保证进程轮转执行。实验中通过 RR_enqueue/RR_pick_next/RR_proc_tick 实现核心逻辑。

对应 OS 原理: 时间片轮转调度算法, 核心是“公平性”与“响应时间”的平衡, 通过时间片限制进程占用 CPU 的时长, 避免单个进程长期占用 CPU。

关系与差异:

关系: 实验严格遵循 RR 算法的核心规则 (时间片、轮转、公平性), 是 OS 原理的最小可运行实现;

差异: OS 原理中的 RR 支持动态调整时间片、优先级加权时间片, 而实验中时间片固定, 未实现优先级扩展, 简化了复杂的参数配置逻辑。

3. Stride 调度算法

实验知识点含义：基于斜堆实现优先级调度，通过 stride（步长）和 pass（步幅， $\text{pass}=\text{BIG_STRIDE}/\text{priority}$ ）控制进程执行频率，优先选择 stride 最小的进程，执行后更新 stride，保证高优先级进程获得更多 CPU 时间。实验中通过斜堆的插入 / 删除 / 堆顶查询实现核心逻辑。

对应 OS 原理：公平共享调度算法（Fair Scheduling），核心是按优先级比例分配 CPU 资源，避免低优先级进程饥饿。

关系与差异：

关系：实验是 Stride 算法的经典实现，严格遵循“步长更新”“优先选最小 stride”的核心规则；

差异：OS 原理中的 Stride 算法会处理 stride 溢出、优先级动态调整、多核负载均衡等问题，实验中仅通过取模避免溢出，未实现多核支持，简化了工程复杂度。

4. 运行队列的多数据结构支持

实验知识点含义：run_queue 同时支持链表（RR）和斜堆（Stride），链表满足 RR 的 $O(1)$ 插入 / 删除需求，斜堆满足 Stride 的 $O(\log N)$ 优先级查询需求。实验中 RR 使用 run_list，Stride 使用 lab6_run_pool。

对应 OS 原理：调度队列的底层数据结构选择，需匹配算法的性能需求（如链表适合 FIFO/RR，堆适合优先级调度，红黑树适合复杂优先级调度）。

关系与差异：

关系：实验精准匹配了数据结构与算法的性能需求，是 OS 原理中“数据结构为算法服务”的典型体现；

差异：OS 原理中调度队列会结合缓存友好性、多核并发等因素选择数据结构，实验中仅实现链表和斜堆，未考虑缓存和多核并发。

5. 调度触发机制（need_resched 标志）

实验知识点含义：时钟中断中仅设置 need_resched 标志，不直接调用 schedule，待中断返回、系统调用返回等“安全点”再检查标志并触发调度。实验中 RR_proc_tick/Stride_proc_tick 均通过该标志触发调度。

对应 OS 原理：操作系统的“延迟调度”机制，核心是避免在中断上下文、临界区等不安全场景切换进程，保证系统稳定性。

关系与差异：

关系：实验严格遵循“延迟调度”的核心规则，是 OS 原理中调度触发机制的最小实现；

差异：OS 原理中 `need_resched` 会结合多核、抢占阈值等因素，实验中仅实现单核场景的标志检查，简化了多核同步逻辑

6. 未涉及知识点

为降低实现难度，实验聚焦基础核心逻辑，未覆盖 OS 原理：

实时调度算法（如 RM/EDF）是实时系统的核心，能保障关键任务截止时间，适用于工业控制、自动驾驶等场景。实验未覆盖的原因是聚焦通用调度算法，未涉及实时性需求，且实时调度需结合硬件定时器、中断优先级等底层机制，超出实验范围。多核调度与负载均衡也是关键知识点，OS 原理中需为每个 CPU 核心维护独立调度队列，实现进程迁移平衡负载；实验仅支持单核，`run_queue` 为全局变量，简化了多核并发问题。

调度算法的性能评估指标（吞吐量、响应时间、周转时间）是评价算法优劣的核心依据，OS 原理中需量化分析这些指标；实验仅通过 `make grade` 验证正确性，未实现量化统计，简化了评估逻辑。动态优先级调度能根据进程行为（IO 密集 / CPU 密集）调整优先级，平衡系统性能；实验中优先级为静态配置，未实现动态调整。